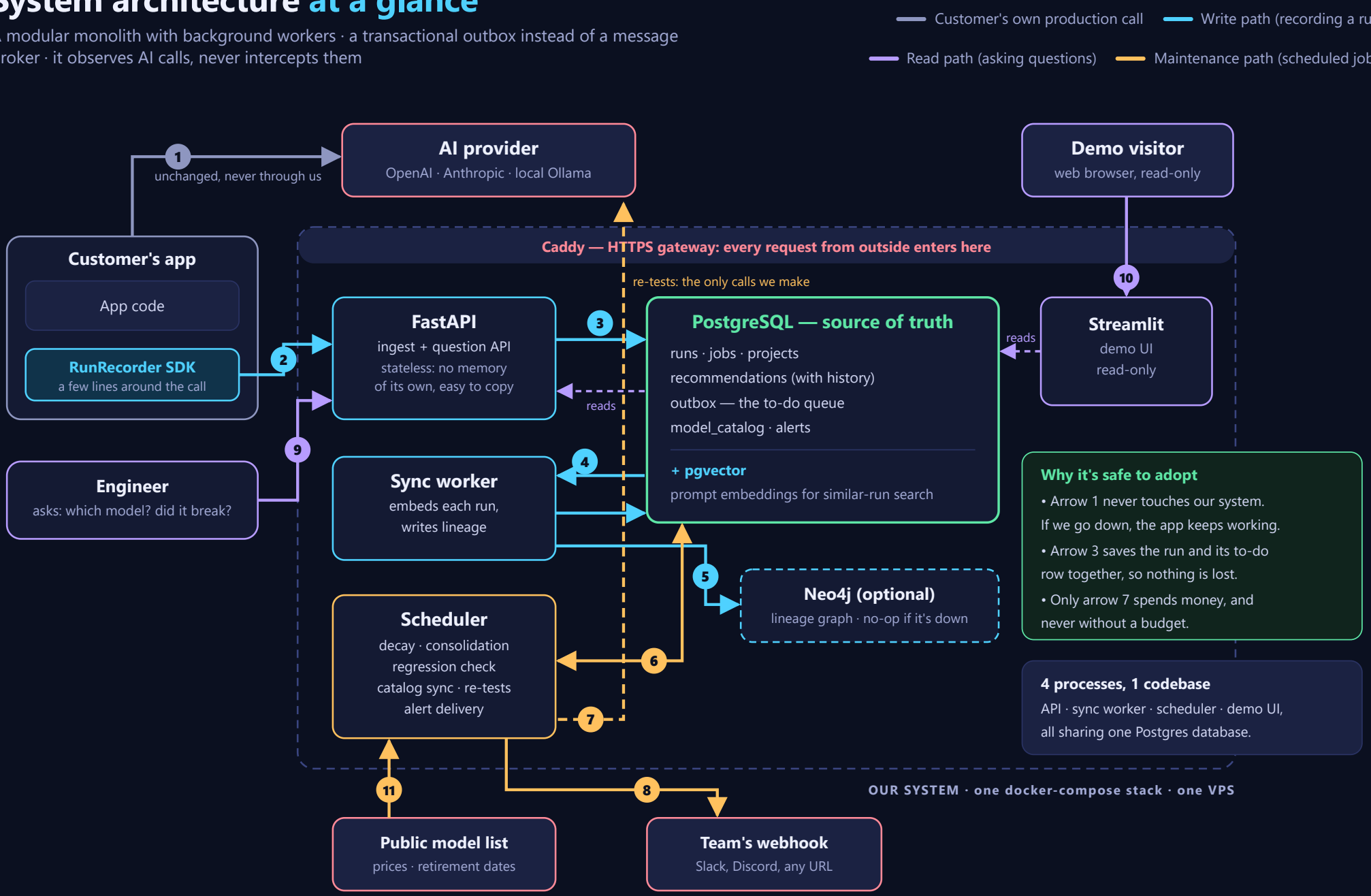
System architecture at a glance

A modular monolith with background workers · a transactional outbox instead of a message broker · it observes AI calls, never intercepts them



Numbers match the walkthrough on page 2

READING THE DIAGRAM

What each numbered arrow means

1 The production call, unchanged
The customer's app calls OpenAI or Anthropic exactly as before. Our system is never in this path, so it can't slow down or break their product.

2 The SDK sends us a copy
After the call returns, `RunRecorder` sends the prompt, model, exact version, tokens, cost, time taken and output to `POST /v1/runs`, in the background. Adds under 5 ms.

3 One transaction: the run and its to-do row
The API checks pass/fail, then saves the run and an **outbox** row in a single database transaction, and replies `202 Accepted`. Either both are saved or neither is.

4 The worker claims to-do rows
The sync worker picks up outbox rows with `SELECT … FOR UPDATE SKIP LOCKED`, so two workers never grab the same row.

5 Embeddings and lineage are written
It turns each prompt into an embedding (for similar-run search in `pgvector`) and writes the job → prompt → model → outcome links to Neo4j. Run becomes `INDEXED` within ~30 s.

6 Scheduled maintenance on the data
Decay recomputes evidence weights, consolidation merges runs into recommendations, and the regression check compares pass rates before and after each version change.

7 Re-tests: the only calls we make
Sampled past runs are replayed on a new or candidate model, within a monthly budget, and only for jobs whose consent flag allows it. Started automatically when a new model appears.

8 Alerts pushed to the team
When a regression is found, an alert is posted to the project's webhook (Slack, Discord or any URL), retried if it fails, and stays visible from the API.

9 The engineer asks questions
Which model should this job use? Did an upgrade break something? Why did the recommendation change? How much does each job cost? Answers in under 500 ms.

10 Demo visitors look, but can't spend
Anyone with the public URL sees recommendations, evidence and alerts in the read-only Streamlit UI. Only team keys can start re-tests.

11 The model catalog stays current
Once a day the scheduler reads a public model list for prices and retirement dates, marks models `ACTIVE`, `DEPRECATED` or `RETIRED`, and warns before a recommended model shuts down.

The one-sentence version: the app works as normal, a copy of every run is saved and checked, background jobs keep the evidence fresh, and the engineer gets answers backed by real runs.

HOW WORK FLOWS

Three independent paths, connected only through the database

A slow re-test never slows down recording, and a burst of new runs never slows down answers.

Write path · every model call

1. App calls the provider (unchanged)
2. SDK sends the run in the background
3. API validates it (bad data → `422`)
4. Pass/fail check: answer key, tests in a sandbox, or schema
5. Run + outbox row saved in one transaction → `202`
6. Worker embeds it and writes lineage

Read path · when someone asks

1. Load the job's active runs
2. Weight each by freshness (decay)
3. Group by model; drop retired models
4. Check there's enough evidence
5. Pick the lowest cost per success within 2 points of the best pass rate
6. Return the verdict with the failing runs

Never writes anything.

Maintenance path · scheduled

- **Decay:** recompute weights; archive below 0.05
- **Consolidation:** runs → one recommendation per job, sources kept
- **Regression check:** two-proportion test across each version change
- **Catalog sync:** retirements, prices, new models
- **Re-tests:** sampled, budgeted, consent-gated
- **Alerts:** deliver to the webhook

Life of one run

`CAPTURED` → `CHECKED` → `INDEXED` → `CONSOLIDATED` → `ARCHIVED` failure: `INGESTION_FAILED`

Saved → pass/fail resolved (or `UNKNOWN` if the check can't run) → searchable → counted in a recommendation → faded below 0.05 weight. **Archived runs are never deleted**; they just stop counting. A run that fails indexing 5 times is marked failed, but the run itself is still saved.

How a recommendation is computed

```
freshness  R(t) = e^(-t / S)          t = days since the run
S = importance × 30 × version_factor    (0.25 once a version is old)

weighted pass rate = Σ(R × passed) / Σ R
cost per success   = mean cost / weighted pass rate
answer = lowest cost per success, among models within
         2 points of the best pass rate, never RETIRED
```

How a regression is caught

```
before vs after a model version change:
  two-proportion test on pass rates
  flag if  p < 0.05  AND  drop ≥ minimum

then: write alert → push to webhook
      supersede the recommendation
      (old one kept, with date + reason)
```

WHY IT'S BUILT THIS WAY

Five architectural decisions

DECISION	WHAT IT MEANS	WHY
1. Observe, never intercept	Our system sits beside the customer's AI calls, not in front of them like a proxy. The SDK sends a copy of each run in the background.	If our system goes down, the customer's app keeps working. That's what makes it safe to adopt.
2. Modular monolith	One codebase with clear modules (`checks` , `storage` , `pipeline` , `recommend` , `retest` , `catalog` , `insights` , `alerts`), run as 4 processes.	Microservices are overkill for a 2-person, 12-week project. Modules keep it organised and can be split out later.
3. Transactional outbox, no broker	The run and its to-do row are saved in one transaction; the worker claims rows with `SKIP LOCKED` .	Nothing is lost or processed twice, with no Kafka or RabbitMQ to run. The database is the queue.
4. Three independent paths	Write, read and maintenance share only the database.	Each can be slow or fail without dragging the others down.
5. Swappable storage	`VectorStoreClient` and `GraphStoreClient` interfaces: `pgvector` now, Qdrant possible later; Neo4j optional with a no-op fallback.	A database can change without touching the logic, and the graph can fail without breaking anything.

Technology, layer by layer

API	Python · FastAPI · Pydantic
Database	PostgreSQL · SQLAlchemy · Alembic migrations
Similar-run search	`pgvector` + local `all-MiniLM-L6-v2` embeddings ($0)
Lineage graph	Neo4j, optional (hosted free tier proposed)
Statistics	`scipy.stats`
Scheduling	APScheduler, in-process
Sandbox	Docker container, no network, time/memory limits
Deploy	docker-compose · one VPS · Caddy HTTPS

What it is not

- **Not microservices:** one codebase, four processes.
- **Not serverless:** the scheduler must run 24/7, so free tiers that sleep won't work.
- **Not a proxy or router:** it never sits in the request path.
- **Not an AI agent:** no model makes decisions. Verdicts are arithmetic and statistics. AI is used only for embeddings, re-tests and an optional summary.
- **Not a message-broker system:** no Kafka or RabbitMQ; the outbox table is the queue.

THE MEMORY SIDE · AND THE WORDS USED

How it works as a memory system

MEMORY IDEA	WHERE IT LIVES	WHAT IT DOES
Episodic memory	`runs` table	"What happened once": one recorded run with its own pass/fail.
Semantic memory	`recommendations` table	"What is generally true": one conclusion per job, linked to every run behind it.
Forgetting	Decay job, `R(t) = e^(-t/S)`	Old evidence fades; evidence about old model versions fades ~4× faster. Nothing is deleted.
Associative recall	`pgvector` embeddings	Finds past runs with similar prompts.
Relational memory	Neo4j lineage graph	Traces job → prompt → model version → outcome.
Revising beliefs	Supersession (`superseded_by_id`)	A changed conclusion replaces the old one but keeps it, with the date and reason.

TERM	PLAIN MEANING	TERM	PLAIN MEANING
SDK	Small Python package the engineer installs; wraps their AI call and sends us a copy.	Embedding	A list of numbers representing a prompt's meaning, so similar prompts can be found.
Outbox	A to-do table saved together with each run, so background work is never lost.	Stateless API	Keeps nothing in memory between requests, so extra copies can run side by side.
SKIP LOCKED	Database feature that lets several workers take different rows without clashing.	Webhook	A URL we send a message to when something happens, e.g. a Slack channel.
pgvector	Postgres add-on for searching embeddings; replaces a separate vector database.	Caddy	The front door: gives the site HTTPS automatically and forwards requests inside.
Decay	Gradually lowering how much an old run counts.	docker-compose	One file that starts every part of the system together, locally or on the server.
Supersession	Replacing a recommendation while keeping the old one and why it changed.	Sandbox	A throwaway, locked-down container for running AI-written code safely.

Source: docs/Technical_Implementation_Plan.md (v3) and docs/PRD.md (v3)